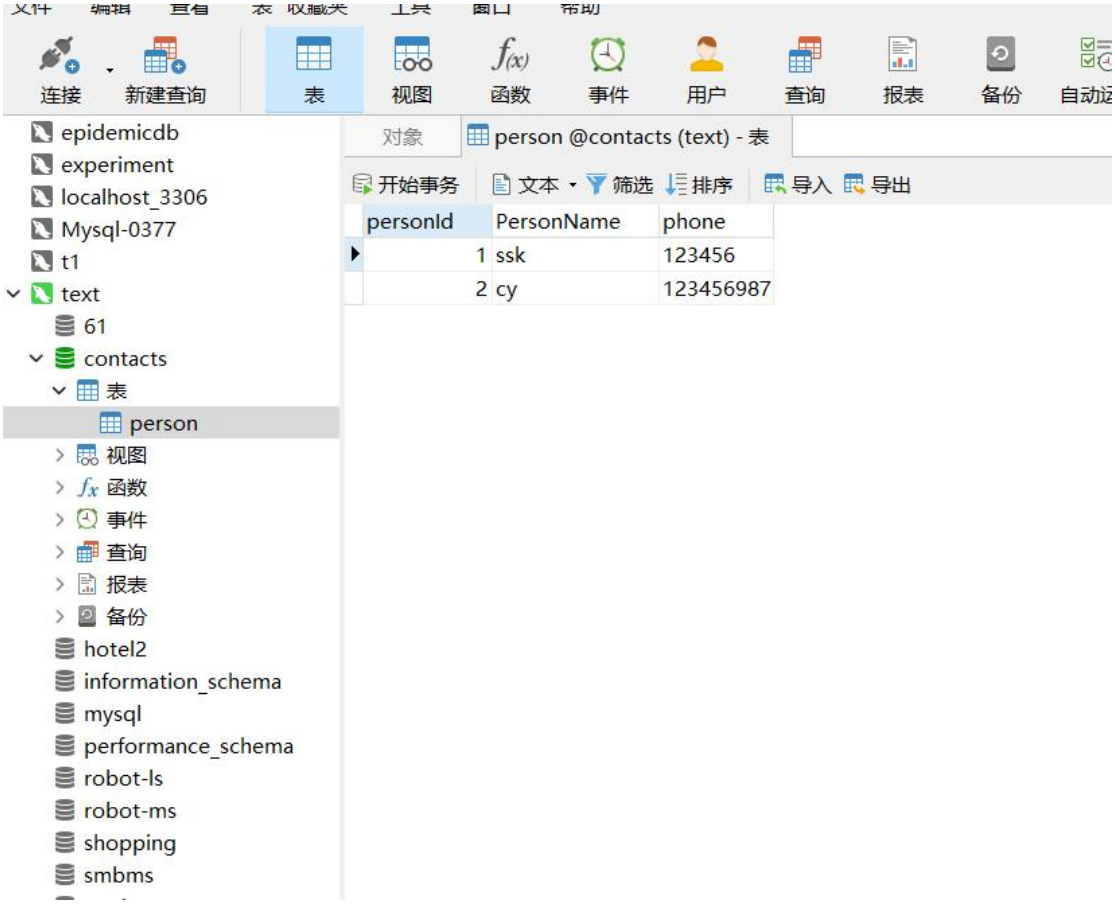
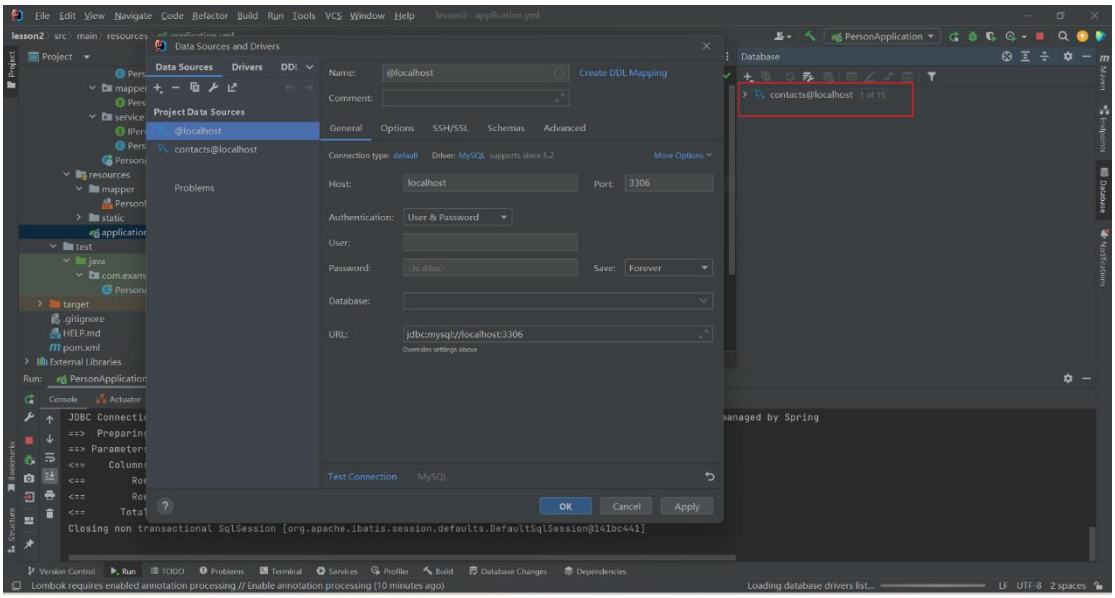
作业三 服务提供者

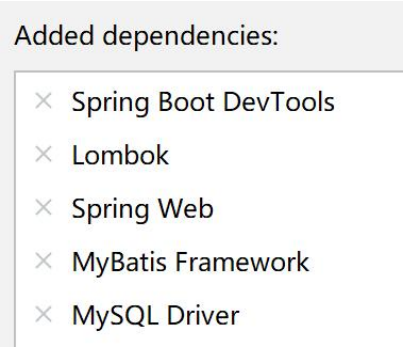
1. 使用 MyBatis 实现数据库查询

1.1. 新建数据库，并创建表





1.2. 新建 Spring Boot 项目



3.pom.xml：添加依赖如下

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.2</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>

    <groupId>com.example</groupId>
    <artifactId>springBootSelectDemo</artifactId>
```

```
<version>0.0.1-SNAPSHOT</version>
<name>springBootSelectDemo</name>
<description>springBootSelectDemo</description>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.mybatis.spring.boot</groupId>
        <artifactId>mybatis-spring-boot-starter</artifactId>
        <version>3.0.3</version>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
```

```
        <dependency>
            <groupId>org.mybatis.spring.boot</groupId>

<artifactId>mybatis-spring-boot-starter-test</artifactId>
            <version>3.0.3</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
        </plugins>
    </build>

</project>
```

application.yml

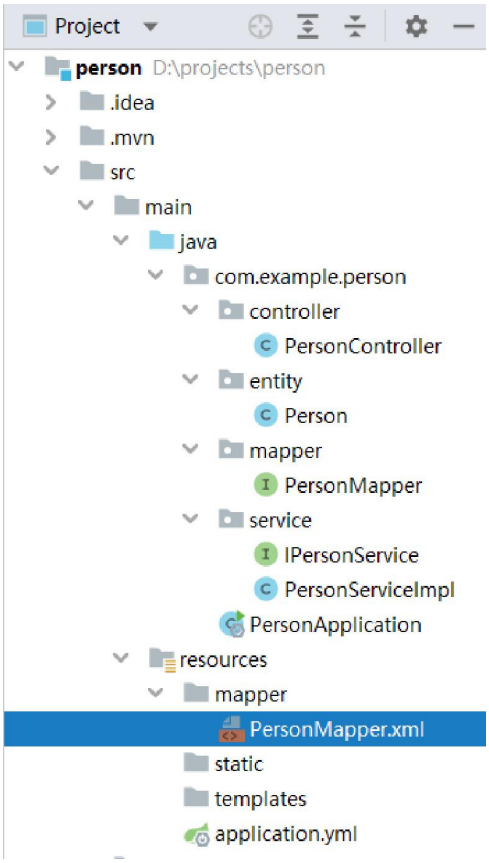
```
server:
  port: 80
spring:
  application:
    name: person
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    url: jdbc:mysql://localhost:3306/contacts
    username: root
    password: root
mybatis:
  mapper-locations: classpath*:mapper/*Mapper.xml
  configuration:
```

```
    map-underscore-to-camel-case: true
    log-impl: org.apache.ibatis.logging.stdout.StdOutImpl
  type-aliases-package: com.example.person.entity
```

自动生成的 PersonApplication.java

```
package com.example.person;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PersonApplication {

    public static void main(String[] args) {
        SpringApplication.run(PersonApplication.class, args);
    }

}
```

分别创建包 entity、mapper、service、controller 和文件



Person.java

```
package com.example.person.entity;

import lombok.Data;

@Data
public class Person {
    private Integer personId;
    private String personName;
    private String phone;
}
```

PersonMapper.java

```
package com.example.person.mapper;

import com.example.person.entity.Person;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Select;

import java.util.List;
```

```
@Mapper
public interface PersonMapper {
//    @Select("SELECT * FROM person")
    List<Person> getPersonList();
}
```

PersonMapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
        PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.example.person.mapper.PersonMapper">
    <select id="getPersonList" resultType="Person">
        SELECT * FROM person
    </select>
</mapper>
```

测试 PersonApplicationTests.java

```
package com.example.person;

import com.example.person.mapper.PersonMapper;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class PersonApplicationTests {
    @Autowired
    private PersonMapper personMapper;

    @Test
    void testGetPersonList() {
        personMapper.getPersonList().forEach(System.out::println);
    }

    @Test
    void contextLoads() {
    }

}
```

IPersonService.java

```
package com.example.person.service;

import com.example.person.entity.Person;

import java.util.List;

public interface IPersonService {
    List<Person> getPersonList();
}
```

PersonServiceImpl.java

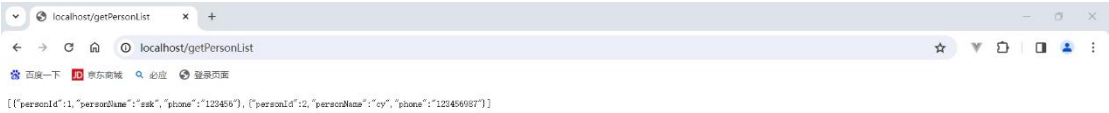
```
package com.example.person.service;

import com.example.person.entity.Person;
import com.example.person.mapper.PersonMapper;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class PersonServiceImpl implements IPersonService{
    @Autowired
    private PersonMapper personMapper;

    @Override
    public List<Person> getPersonList() {
        return personMapper.getPersonList();
    }
}
```

PersonController.java

```
package com.example.person.controller;

import com.example.person.entity.Person;
import com.example.person.service.PersonServiceImpl;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;
```

```
import java.util.List;

@RestController
public class PersonController {
    @Autowired
    private PersonServiceImpl personService;

    @RequestMapping(value = "/getPersonList", method =
RequestMethod.GET)
    public List<Person> getPersonList() {
        return personService.getPersonList();
    }
}
```

运行结果



代码分享：https://gitee.com/zhang-jike-kang/lesson2.git

2. 通过 RestTemplate 实现服务调用

1. 配置 pom 文件，添加 spring-boot-starter-web / spring-cloud-starter-alibaba-nacos-discovery / springboot-starter-actuator / spring-cloud-starter-loadbalancer 四个依赖
2. 对 application.yml，进行配置

```yaml
server:
  port: 9003
spring:
  application:
    name: consumer9003
  cloud:
    nacos:
      discovery:
        server-addr: 127.0.0.1:8848
        service: consumer9003
management:
  endpoints:
    web:
      exposure:
        include: '*'
```

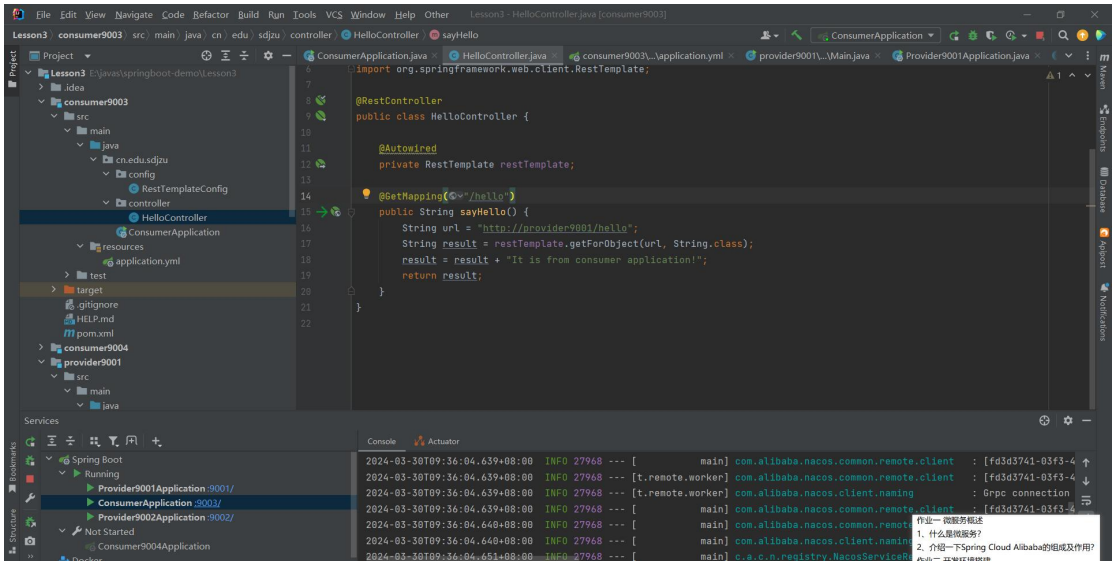
3. 创建 ConsumerApplication，使用注释@SpringBootApplication，注释 *@EnableDiscoveryClient* 可不添加

4. 创建 controller.HellowControllrt 类，注意 url 使用服务名

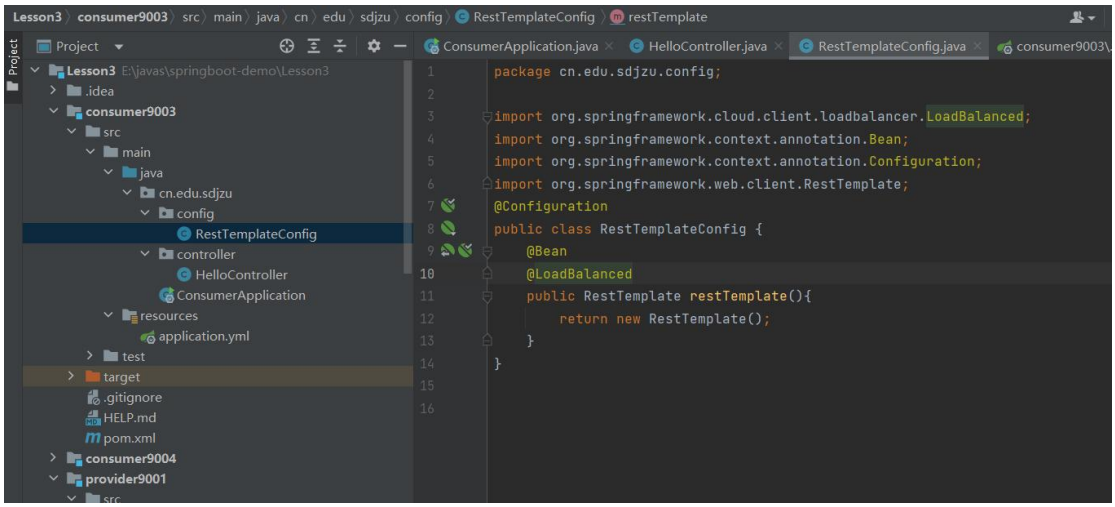
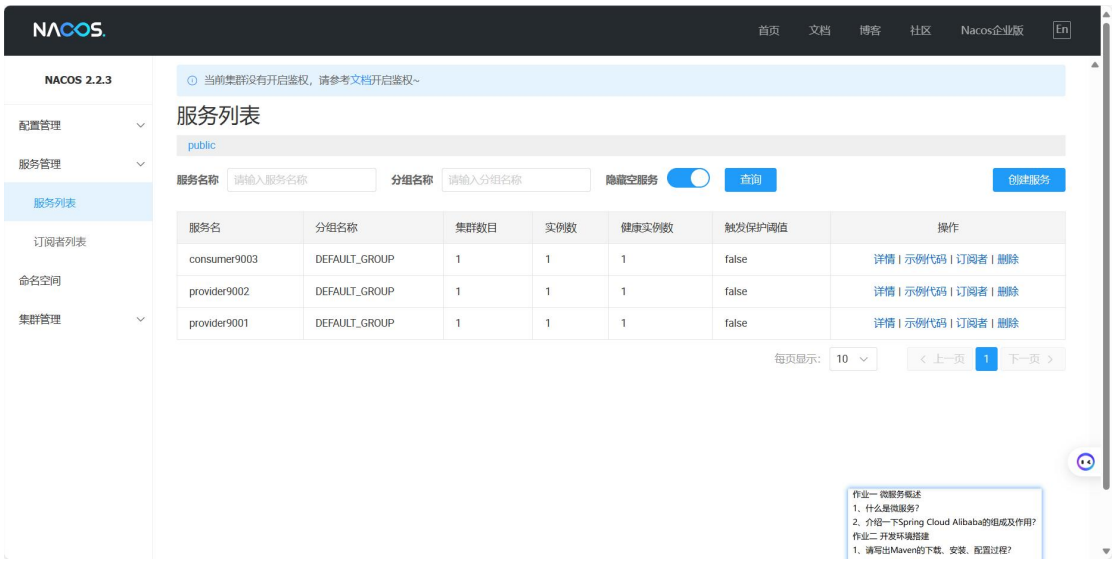
注：类中的@RestController 相当于@ResponseBody 和@Controller 这两个注解的集合。

@RestController 是一个用于定义 RESTful Web 服务的控制器的特殊注解。

@Controller 是 Spring 框架中用于标识一个类是控制器组件的注解。



5. 创建 config.RestTemplateConfig 类



6. 打开 Nacos，运行 9001，9002，9003